15. Functions

Created by Li Huanyu, last modified by suncaper on Mar 09, 2026

Introduction

In the previous lessons, we learned how to use loops to repeat actions over collections of data. But what if you need to use the exact same data-cleaning logic in three different scripts? Copying and pasting your code is dangerous—if you find a bug, you have to remember to fix it in every single place you pasted it.

A **Function** is a reusable block of code that only runs when it is called. You can pass data into it (inputs), let it perform its operations, and have it send data back out (outputs).

In Data Engineering, functions are the building blocks of clean, modular ETL pipelines. Instead of writing a massive 1,000-line script, Data Engineers break their pipelines down into logical, bite-sized functions like `extract_data()`, `clean_nulls()`, and `load_to_database()`.

In this lesson, we will cover:

- Defining a function using the `def` keyword.
- Passing inputs using **Parameters**.
- Extracting outputs using the `return` statement.
- Why functions are the prerequisite for PySpark UDFs (User Defined Functions).

Part 1: Defining a Function (`def`)

To create a function in Python, you use the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`. The code block inside the function must be indented.

```
# 1. Defining the function
def print_pipeline_status():
    print("STATUS: ETL Pipeline is currently running...")

# 2. Calling (executing) the function
print_pipeline_status()
print_pipeline_status()
# Output will print the status message twice.
```

Part 2: Parameters (Passing Data In)

Functions become powerful when they can process dynamic data. A **Parameter** is a variable listed inside the parentheses in the function definition. When you call the function, the actual value you pass to it is called an **Argument**.

```
# 'table_name' is the parameter
def truncate_table(table_name):
    print("Executing SQL: TRUNCATE TABLE " + table_name + ";")

# "users" and "sales" are the arguments
truncate_table("users")  # Output: Executing SQL: TRUNCATE TABLE users;
```

```
truncate_table("sales")  # Output: Executing SQL: TRUNCATE TABLE sales;
```

You can require multiple parameters by separating them with commas:

```
def load_data(source_file, destination_table):
    print("Moving data from " + source_file + " to " + destination_table)
```

Part 3: Return Values (Getting Data Out)

If you want your function to calculate something and give the answer back to your main script, you must use the `return` keyword.

Printing vs. Returning: `print()` just shows text on the screen for the human. `return` hands the actual data object back to Python so you can store it in a variable and use it later in your pipeline.

```
# A function that performs an operation and returns a result
def calculate_tax(order_total, tax_rate):
    tax_amount = order_total * tax_rate
    return tax_amount

# We must capture the returned data in a variable!
final_tax = calculate_tax(100.0, 0.08)

print("Tax to be collected: $" + str(final_tax)) # Output: Tax to be collected: $8.0
```

Part 4: The "Big Data" Context (Why do we care?)

- **The DRY Principle (Don't Repeat Yourself):** If you are writing the same logic more than twice, it needs to be a function. This makes your ETL pipelines easier to read, test, and debug.
- **PySpark UDFs (User Defined Functions):** While Spark SQL has many built-in functions, you will eventually encounter custom business logic (e.g., "Calculate a dynamic discount based on 5 different user parameters"). To do this in Spark, you write a standard Python `def` function and register it as a UDF, which Spark then distributes across the entire cluster to run on millions of rows.
- **Testing and CI/CD:** You cannot automatically test a massive, unstructured script. Data Engineers write functions so they can write automated "Unit Tests" to verify that `clean_currency("$100")` actually returns `100.0` before deploying the pipeline to production.

Part 5: Hands-on Exercise (Data Cleaning Function)

Let's simulate a common data cleaning task. We constantly receive messy currency strings from an external API (like `"$1,250.99"`). If we try to cast this directly to a float, Python will throw a `ValueError`. We need to build a reusable function to clean it.

1. Open PyCharm.
2. Create a new file named `function_practice.py`.
3. Type the following code:

```
# ------------------------------------------------------------
# Big Data Simulation: Modular Data Cleaning
```

```
# ---------------------------------------------------------

"""
Scenario: We need a reusable function to clean messy currency strings
so they can be safely loaded into a numerical database column.
"""

# 1. DEFINE: Create our reusable transformation function
def clean_currency_string(raw_price):
    # Step A: Remove whitespace from the edges
    step_1 = raw_price.strip()

    # Step B: Remove the dollar sign
    step_2 = step_1.replace("$", "")

    # Step C: Remove any commas used for thousands
    step_3 = step_2.replace(",", "")

    # Step D: Cast the clean string to a float
    final_float = float(step_3)

    # Return the math-ready number back to the pipeline
    return final_float


# 2. EXECUTE: Test our function on messy pipeline data
raw_input_1 = "  $1,450.75 "
raw_input_2 = "$99.99"

print("--- Starting Transformation ---")

# Call the function and save the returned results
clean_price_1 = clean_currency_string(raw_input_1)
clean_price_2 = clean_currency_string(raw_input_2)

# Verify we can now do math with the results!
total_revenue = clean_price_1 + clean_price_2

print("Clean Price 1: " + str(clean_price_1))
print("Clean Price 2: " + str(clean_price_2))
print("Total Revenue: " + str(total_revenue))


Run the code. Output:

Plaintext

--- Starting Transformation ---
Clean Price 1: 1450.75
```

```
Clean Price 2: 99.99
Total Revenue: 1550.74
```

What happens?

- We encapsulated four distinct data-cleaning steps inside a single function named `clean_currency_string()`.
- We called the function twice with completely different raw inputs.
- The function processed the strings, stripped the bad characters, converted them to floats, and used `return` to hand the clean data back to us.
- Because the returned data was numerical, we successfully performed math (`total_revenue`) without crashing.

Summary

- **`def`** is used to define a new function.
- **Parameters** act as placeholders for the data you pass *into* the function.
- **`return`** is used to pass the final calculated data *out of* the function so your script can use it.
- **Data Pipelines depend on Functions:** They transform spaghetti code into modular, testable, and reusable blocks, which is the absolute standard for professional Data Engineering.

No labels